

CSE 1203

Object Oriented Programming [C++]

Chapter 4: Advanced Topics-1

Learning Objectives

2

To know about:

- STL Array
- STL Pair & Tuple
- STL Vector

STL in C++

STL: Standard Template Library

It consists of three components

- i) **Container** : where data stored like array, link list, stack etc
- ii) **Iterator**: move forward/backward in the container (different containers have different Iterator)
- iii) **Algorithm**: different algorithms are available like searching, sorting etc



Extremly useful for
competative programming

STL in C++ (Array)

The array is a collection of homogeneous objects and this array container is defined for constant size arrays or (static size).

In order to utilize arrays, we need to include the array header: `#include <array>`

```
#include<iostream>
#include<array> // for array of STL
using namespace std;
int main() {

    // construction uses aggregate initialization
    // double-braces required
    array<int, 5> ar1{{3, 4, 5, 1, 2}};
    array<int, 5> ar2 = {1, 2, 3, 4, 5};
    array<string,3> ar3 = {"Raj", "Dha","Chi"};

    cout << "Sizes of arrays are" << endl;
    cout << ar1.size() << endl;
    cout << ar2.size() << endl;
    cout << ar3.size() << endl;

    cout << "\nInitial ar1 : ";
    for (auto i : ar1)
        cout << i << ' ';
}
```

STL in C++ (Array)

Member Functions of Array Template

Following are the important and most used member functions of array template.

i) **at** function

This method returns value in the array at the given range. If the given range is greater than the array size, `out_of_range` exception is thrown. Here is a code snippet explaining the use of this operator :

ii) **[]** Operator

The use of operator `[ ]` is same as it was for normal arrays. It returns the value at the given position in the array. Example : In the above code, statement `cout << array1[5];` would print 6 on console as 6 has index 5 in array1.

iii) **front()** function

This method returns the first element in the array.

iv) **back()** function

This method returns the last element in the array. The point to note here is that if the array is not completely filled, `back()` will return the rightmost element in the array.

v) **fill()** function

This method assigns the given value to every element of the array,

STL in C++ (Array)

Member Functions of Array Template contd

vi) **swap()** function

This method swaps the content of two arrays of same type and same size.

vii) **empty()** function

This method can be used to check whether the array is empty or not.

Syntax: array_name.empty(), returns true if array is empty else return false.

viii) **size()** function

This method returns the number of element present in the array.

ix) **max_size()** function

This method returns the maximum size of the array.

For array, size() and max_size() will always give the same result.

x) **begin()** function

This method returns the iterator pointing to the first element of the array.

xi) **end()** function

This method returns an iterator pointing to an element next to the last element in the array

STL in C++ (Array Exmample)

```
#include<bits/stdc++.h>
#include<array>
using namespace std ;

int main()
{
    array<int,5>ax;
    ax={10,20,30,40,50};
    cout<<"The array: ";
    for(auto x:ax)
        cout<<x<<" ";
    cout<<endl;
    cout<<"Element at index 3:"<<ax[3]<<endl;
    cout<<"Element at index 3:"<<ax.at(3)<<endl;
    cout<<"Front Element:"<<ax.front()<<endl;
    cout<<"Back Element:"<<ax.back()<<endl;
    cout<<"Size of ax:"<<ax.size()<<endl;
    cout<<"Size of ax:"<<ax.max_size()<<endl;
    cout<<"Address of 1st Element:"<<ax.begin()<<endl;
    cout<<"Address of last Element:"<<ax.end()<<endl;
    ax.fill(7);
    cout<<"The array: ";
    for(auto x:ax)
        cout<<x<<" ";
}
```

STL in C++ (pair class)

Pair is used to combine together two values that may be of different data types. Pair provides a way to store two heterogeneous objects as a single unit. It is basically used if we want to store tuples. The pair container is a simple container defined in `<utility>` header consisting of two data elements or objects.

- The first element is referenced as 'first' and the second element as 'second' and the order is fixed (first, second).
- Pair can be assigned, copied, and compared. The array of objects allocated in a [map](#) or `hash_map` is of type 'pair' by default in which all the 'first' elements are unique keys associated with their 'second' value objects.
- To access the elements, we use variable name followed by dot operator followed by the keyword first or second.

Syntax:

`pair (data_type1, data_type2) Pair_name`

1) make_pair(): This template function allows to create a value pair without writing the types explicitly.

Syntax: `Pair_name = make_pair (value1,value2);`

2) swap: This function swaps the contents of one pair object with the contents of another pair object. The pairs must be of the same type.

Syntax: `pair1.swap(pair2) ;`

```
#include <iostream>
#include<bits/stdc++.h>
#include<array>
//#include<utility>
using namespace std;

int main()
{
    pair<int,string>a;
    pair<int,string>b;
    a=make_pair(100,"Zaman");
    b=make_pair(200,"Aslam");
    //a={200,"Belal"};
    //a.second="Kamal";
    a.swap(b);
    cout<<a.first<<" "<<a.second;

}
```

```
int main() {
    pair<int,pair<int,int>>p;// graph
    p.second.first=5;
}
```

STL in C++ (**tuple** class)

What is a tuple?

A tuple is an object that can hold a number of elements. The elements can be of different data types. The elements of tuples are initialized as arguments in order in which they will be accessed.

Operations on tuple :-

- 1. get()** :- get() is used to access the tuple values and modify them, it accepts the index and tuple name as arguments to access a particular tuple element.
- 2. make_tuple()** :- make_tuple() is used to assign tuple with values. The values passed should be in order with the values declared in tuple.
- 3. swap()** :- The swap(), swaps the elements of the two different tuples.
- 3. tie()** :- copy the values of tuple elements to the corresponding variables

```
#include <iostream>
#include<bits/stdc++.h>
#include<tuple>
//#include<utility>
using namespace std;

int main()
{
    int p; string q;double r;
    tuple<int,string,double>a,b;
    a=make_tuple(100,"Zaman",500.5);
    b=make_tuple(200,"Belal",500.5);
    cout<<get<0>(a);
    //get<1>(a)="Kamal";
    cout<<get<1>(a);
    a.swap(b);
    cout<<get<1>(a);
    tie(p,q,r)=a;
    cout<<p<<q<<r;
}
```

STL : *Vector*

STL in C++ (Vector)

Vector is a dynamic array

SYNTAX for creating a vector is: `vector< object_type > vector_name;`

Member Functions

`push_back()` is used for inserting an element at the end of the vector.

`insert(itr, element)` method inserts the element in vector before the position pointed by iterator `itr`.

`pop_back()` is used to remove the last element from the vector. It reduces the size of the vector by one.

`erase(itr_pos)` removes the element pointed by the iterator **itr_pos**.

`reserve(value)` method allocates capacity

`swap ()` method interchanges value of two vectors.

`clear()` method clears the whole vector, removes all the elements from the vector but do not delete the vector.

`size ()` This method returns the size of the vector.

`empty()` method returns true if the vector is empty else returns false.

`resize(n)` method resizes the vector to **n** elements.

`capacity()` method returns the number of elements that can be inserted in the vector.

The sizes becomes double if it is full.

`at()` method works same in case of vector as it works for array.

`front()` returns the element at the front of the vector (i.e. leftmost element).

`back()` returns the element at the back of the vector (i.e. rightmost element).

STL in C++ (Vector)

Member Functions (contd)

begin() – Returns an iterator pointing to the first element in the vector

end() – Returns an iterator pointing to the theoretical element that follows the last element in the vector

rbegin() – Returns a reverse iterator pointing to the last element in the vector (reverse beginning). It moves from last to first element

rend() – Returns a reverse iterator pointing to the theoretical element preceding the first element in the vector (considered as reverse end)

cbegin() – Returns a constant iterator pointing to the first element in the vector.

cend() – Returns a constant iterator pointing to the theoretical element that follows the last element in the vector.

crbegin() – Returns a constant reverse iterator pointing to the last element in the vector (reverse beginning). It moves from last to first element

crend() – Returns a constant reverse iterator pointing to the theoretical element preceding the first element in the vector (considered as reverse end)

```
vector<int>bx={10,20,30};  
vector<int>::const_iterator p=bx.cbegin()+1;  
    *p=90;  
    cout<<"p="<<*p<<endl;
```

Produces Error as p is constant pointer so *p=90 is illegal

STL in C++ (Iterator)

Iterators are used to point at the memory addresses of [STL](#) containers. They are primarily used in sequences of numbers, characters etc. They reduce the complexity and execution time of the program.

Operations of iterators :-

1. begin() :- This function is used to return the **beginning position** of the container.

2. end() :- This function is used to return the **after end position** of the container.

3. advance() :- This function is used to **increment the iterator position** till the specified number mentioned in its arguments.

Output

The vector elements are : 1 2 3 4 5

The position of iterator after advancing is : 4

```
#include<iostream>
#include<iterator> // for iterators
#include<vector> // for vectors
using namespace std;
int main()
{
    vector<int> ar = { 1, 2, 3, 4, 5 };
    // Declaring iterator to a vector
    vector<int>::iterator ptr;
    // Displaying vector elements using begin()
    and end()
    cout << "The vector elements are : ";
    for (ptr = ar.begin(); ptr < ar.end(); ptr++)
        cout << *ptr << " ";
    // Using advance() to increment iterator position
    // points to 4
    advance(ptr, 3);

    // Displaying iterator position
    cout << "The position of iterator after
    advancing is : ";
    cout << *ptr << " ";

    return 0;
}
```

STL in C++ (Iterator)

4. next() :- This function **returns the new iterator** that the iterator would point after **advancing the positions** mentioned in its arguments.

5. prev() :- This function **returns the new iterator** that the iterator would point **after decrementing the positions** mentioned in its arguments.

Output

The position of new iterator using next() is : 4
The position of new iterator using prev() is : 3

```
#include<iostream>
#include<iterator> // for iterators
#include<vector> // for vectors
using namespace std;
int main()
{
    vector<int> ar = { 1, 2, 3, 4, 5 };
    // Declaring iterators to a vector
    vector<int>::iterator ptr = ar.begin();
    vector<int>::iterator ftr = ar.end();

    // Using next() to return new iterator
    // points to 4
    auto it = next(ptr, 3);
    // Using prev() to return new iterator
    // points to 3
    auto it1 = prev(ftr, 3);
    // Displaying iterator position
    cout << "The position of new iterator using next() is
: ";
    cout << *it << " ";
    cout << endl;
    // Displaying iterator position
    cout << "The position of new iterator using
prev() is : ";
    cout << *it1 << " ";
    cout << endl;
    return 0;
}
```

STL in C++ (Iterator)

6. inserter() :- This function is used to **insert the elements at any position** in the container. It accepts **2 arguments, the container and iterator to position where the elements have to be inserted.**

```
#include<iostream>
#include<iterator> // for iterators
#include<vector> // for vectors
using namespace std;
int main()
{
    vector<int> ar = { 1, 2, 3, 4, 5 };
    vector<int> ar1 = {10, 20, 30};

    // Declaring iterator to a vector
    vector<int>::iterator ptr = ar.begin();

    // Using advance to set position
    advance(ptr, 3);

    // copying 1 vector elements in other using
    inserter()
    // inserts ar1 after 3rd position in ar
    copy(ar1.begin(), ar1.end(), inserter(ar,ptr));

    // Displaying new vector elements
    cout << "The new vector after inserting elements is :
";
    for (int &x : ar)
        cout << x << " ";

    return 0;
}
```

Output

The new vector after inserting elements is : 1 2 3 10 20 30 4 5

STL in C++ (Iterator)

7. sort() :- Sort the vector elements in ascending or descending order

```
#include<iostream>
#include<iterator> // for iterators
#include<vector> // for vectors
using namespace std;

void Display(vector<int>bx){
    for(auto x:bx){
        cout<<x<<" ";
    }
    cout<<endl;
}

int main()
{
    vector<int>ax={1,2,3,4,5},bx={100,200,300};
    vector<int>::iterator p;
    p=ax.begin();
    advance(p,2);
    copy(bx.begin(),bx.end(),inserter(ax,p));
    Display(ax);
    sort(ax.rbegin(),ax.rend());
    Display(ax);
    //cout<<" value="<<*p<<endl;
}
```

Output

The new vector after inserting elements is : 1 2 3 10 20 30 4 5

STL in C++ (vector Example)

```
#include<bits/stdc++.h>
#include<array>
using namespace std ;
void Display(vector<int> &p){
    for(int i=0;i<p.size();i++)
        cout<<p[i]<<" ";
    cout<<endl;
}
int main()
{
    vector<int>v{10,20,30,40,50};
    vector<int>::iterator it;
    vector<int>k;
    cout<<"The vector: ";
    for(auto x:v)
        cout<<x<<" ";
    cout<<endl;
    v.push_back(60);
    k.pop_back();
    it=v.begin();
    v.insert(it,566);
    v.insert(it+2,5,15);
    cout<<endl;
```

```
    cout<<"The vector: ";
    for(auto x:v)
        cout<<x<<" ";

    k.reserve(31); //without reserve
    capacity is always doubled
    for(int i=1;i<=32;i++){
        k.push_back(i);
        cout<<" size="<<k.size()<<"
capacity="<<k.capacity()<<endl;
    }

    cout<<"k vector:";
    Display(k);
}
```

STL in C++ (**stack** class)

[empty\(\)](#) – Returns whether the stack is empty

[size\(\)](#) – Returns the size of the stack

[top\(\)](#) – Returns a reference to the top most element of the stack

[push\(g\)](#) – Adds the element 'g' at the top of the stack

[pop\(\)](#) – Deletes the top most element of the stack

```
#include <iostream>
#include<bits/stdc++.h>
#include<stack>
#include <algorithm>
//#include<utility>
using namespace std;

int main()
{
    stack<int>st;
    st.push(10);
    st.push(20);
    st.push(30);
    st.push(40);
    st.pop();
    cout<<"front="<<st.top()<<endl;
    cout<<"size="<<st.size()<<endl;
}
```

STL in C++ (**queue** class)

empty() Returns whether the queue is empty. It returns true if the queue is empty otherwise returns false.

size() Returns the size of the queue.

swap() Exchange the contents of two queues but the queues must be of the same data type, although sizes may differ.

front() Returns a reference to the first element of the queue.

back() Returns a reference to the last element of the queue.

push(g) Adds the element 'g' at the end of the queue.

pop() Deletes the first element of the queue.

```
#include <iostream>
#include<bits/stdc++.h>
#include<queue>
#include <algorithm>
using namespace std;

int main()
{
    queue<int>qu;
    qu.push(10);
    qu.push(20);
    qu.push(30);
    qu.push(40);
    qu.pop();
    cout<<"front="<<qu.front()<<endl;
    cout<<"back="<<qu.back()<<endl;
    cout<<"size="<<qu.size()<<endl;
}
```

STL in C++ (**deque** class)

<u>insert()</u>	Inserts an element. And returns an iterator that points to the first of the newly inserted elements.
<u>rbegin()</u>	Returns a reverse iterator which points to the last element of the deque (i.e., its reverse beginning).
<u>rend()</u>	Returns a reverse iterator which points to the position before the beginning of the deque (which is considered its reverse end).
<u>cbegin()</u>	Returns a constant iterator pointing to the first element of the container, that is, the iterator cannot be used to modify, only traverse the deque.
<u>max_size()</u>	Returns the maximum number of elements that a deque container can hold.
<u>push_front()</u>	It is used to push elements into a deque from the front.
<u>push_back()</u>	This function is used to push elements into a deque from the back.
<u>pop_front()</u> and <u>pop_back()</u>	pop_front() function is used to pop or remove elements from a deque from the front. pop_back() function is used to pop or remove elements from a deque from the back.
<u>front()</u> and <u>back()</u>	front() function is used to reference the first element of the deque container. back() function is used to reference the last element of the deque container.
<u>clear()</u> and <u>erase()</u>	clear() function is used to remove all the elements of the deque container, thus making its size 0. erase() function is used to remove elements from a container from the specified position or range.
<u>empty()</u> and <u>size()</u>	empty() function is used to check if the deque container is empty or not. size() function is used to return the size of the deque container or the number of elements in the deque container.

```
#include <iostream>
#include<bits/stdc++.h>
#include<queue>
#include <algorithm>
//#include<utility>
using namespace std;

int main()
{
    deque<int>qu;
    deque<int>::iterator it;
    qu.push_back(10);
    qu.push_back(20);
    qu.push_back(30);
    qu.push_back(40);
    //qu.pop_back();
    cout<<"front="<<qu.front()<<endl;
    cout<<"back="<<qu.back()<<endl;
    for(it = qu.begin();it!=qu.end();it++)
    {
        cout << *it << " ";
    }
}
```

STL in C++ (**list** class)

<u>front()</u>	Returns the value of the first element in the list.
<u>back()</u>	Returns the value of the last element in the list.
<u>push_front(g)</u>	Adds a new element 'g' at the beginning of the list.
<u>push_back(g)</u>	Adds a new element 'g' at the end of the list.
<u>pop_front()</u>	Removes the first element of the list, and reduces size of the list by 1.
<u>pop_back()</u>	Removes the last element of the list, and reduces size of the list by 1.
<u>begin()</u>	begin() function returns an iterator pointing to the first element of the list.
<u>end()</u>	end() function returns an iterator pointing to the theoretical last element which follows the last element.
<u>rbegin() and rend()</u>	rbegin() returns a reverse iterator which points to the last element of the list. rend() returns a reverse iterator which points to the position before the beginning of the list.
<u>cbegin() and cend()</u>	cbegin() returns a constant random access iterator which points to the beginning of the list. cend() returns a constant random access iterator which points to the end of the list.
<u>crbegin() and crend()</u>	crbegin() returns a constant reverse iterator which points to the last element of the list i.e reversed beginning of container. crend() returns a constant reverse iterator which points to the theoretical element preceding the first element in the list i.e. the reverse end of the list.
<u>empty()</u>	Returns whether the list is empty(1) or not(0).
<u>insert()</u>	Inserts new elements in the list before the element at a specified position.

21

<u>erase()</u>	Removes a single element or a range of elements from the list.
<u>assign()</u>	Assigns new elements to list by replacing current elements and resizes the list.
<u>remove()</u>	Removes all the elements from the list, which are equal to given element.
<u>remove_if()</u>	Used to remove all the values from the list that correspond true to the predicate or condition given as parameter to the function.
<u>reverse()</u>	Reverses the list.
<u>size()</u>	Returns the number of elements in the list.
<u>resize()</u>	Used to resize a list container.
<u>sort()</u>	Sorts the list in increasing order.
<u>max_size()</u>	Returns the maximum number of elements a list container can hold.
<u>unique()</u>	Removes all duplicate consecutive elements from the list.
<u>emplace_front()</u> <u>and</u> <u>::emplace_back()</u>	emplace_front() function is used to insert a new element into the list container, the new element is added to the beginning of the list. emplace_back() function is used to insert a new element into the list container, the new element is added to the end of the list.
<u>list::clear()</u>	clear() function is used to remove all the elements of the list container, thus making it size 0.
<u>list::operator=</u>	This operator is used to assign new contents to the container by replacing the existing contents.
<u>list::swap()</u>	This function is used to swap the contents of one list with another list of same type and size.
<u>list splice()</u>	Used to transfer elements from one list to another.

STL in C++ (**list** class)

```
#include <iostream>
#include<bits/stdc++.h>
#include<stack>
#include <algorithm>
#include <iterator>
//#include<utility>
using namespace std;

// a predicate implemented as a function:
bool isOdd(int value)
{ return (value%2)==1; }

//Display list in forward direction
void Display(list<int> g){
    list<int>::iterator it;
    for(it=g.begin();it!=g.end();it++){
        cout<<" "<<*it;
    }
    cout<<endl;
}

//Display list in reverse direction
void DisplayR(list<int> g){
    list<int>::reverse_iterator it;
    for(it=g.rbegin();it!=g.rend();it++){
        cout<<" "<<*it;
    }
    cout<<endl;
}
```

```
int main()
{
    list<int>ar;
    list<int>::iterator p,q;
    ar.push_back(30);
    ar.push_back(21);
    ar.push_back(21);
    ar.push_front(21);
    DisplayR(ar);
    //ar.pop_front();
    Display(ar);
    //ar.pop_back();
    DisplayR(ar);
    cout<<"front="<<ar.front()<<endl;
    cout<<"back="<<ar.back()<<endl;

    //insert before first element
    p=ar.begin();
    ar.insert(p,11);
    Display(ar);

    //insert after last element
    p=ar.end();
    ar.insert(p,27);
    Display(ar);
}
```

STL in C++ (**list** class)

```
//insert before a node 30
p=find(ar.begin(),ar.end(),30);
ar.insert(p,22);
if(p==ar.end())
    cout<<"Not found"<<endl;
else
    cout<<"Found address="<<&p<<endl;
Display(ar);

//insert after a node 30
p=find(ar.begin(),ar.end(),30);
advance(p,1);
ar.insert(p,26);
if(p==ar.end())
    cout<<"Not found"<<endl;
else
    cout<<"Found address="<<&p<<endl;
Display(ar);

//delete 2nd element
p=ar.begin();
ar.erase(++p);
Display(ar);

//delete a specific element 30
p=find(ar.begin(),ar.end(),30);
ar.erase(p);
Display(ar);
```

```
//delete elements within the range
//here delete first four elements
p=ar.begin();
q=ar.begin();
advance(q,1); //error write q=q+4
ar.erase(p,q);
Display(ar);

//remove elements of value=26
ar.remove(28);
Display(ar);

//remove elements with condition
cout<<"Remove if"<<endl;
//ar.remove_if(isOdd);
Display(ar);

//count an element
int mc;
Display(ar);
mc = count (ar.begin(), ar.end(), 22);
cout<<"Count="<<mc<<endl;

//count elements with condition
Display(ar);
mc=count_if(ar.begin(), ar.end(),isOdd);
cout<<"Count odd="<<mc<<endl;
```

STL in C++ (**list** class)

```
//assign elements
cout<<"Assign"<<endl;
list<int> br;
br.assign (7,100); // 7 ints with value
100
Display(br);
br.assign (ar.begin(),ar.end()); // a
copy of first
Display(br);
int myints[]={1776,7};
br.assign (myints,myints+2);
Display(br);           // assigning
from array

//Delete consecutive unique values
cout<<"Unique value deletion"<<endl;
ar.sort();
ar.unique();
Display(ar);
}
```

4

25

THANK YOU